Name Kaushik Vada SID 862441522

CS/EE 147
GPU Architecture and Parallel Programming

Spring 2024

Midterm

Time Allowed: 80 Minutes

Question	Points Possible	Points Scored
1	4	
2	4	
3	4	
4	13	
5	12	
6-9 (MC)	12	
10-17 (T/F)	12	
18	8	
19	12	
20	4	
Total	85	

- Closed book, closed internet
- Allowed 1 page letter-size cheatsheet, front and back
- Calculators allowed, but likely not necessary

Each thread computes 8 elements

thread 0 → 0-7

thread 1 → 8-15

thread 2 → 16-23

Name _____ SID _____

1. [4 points] CUDA Kernel Launch
   You want to launch a kernel function, `kernel()`, with 32 blocks of 256 threads per block. Please write your answer in the provided box.

   a. [2 points] How many warps are in each block?

   Answer: 8 warps

   b. [2 points] How many total threads are in your kernel?

   Answer: 8192 threads

2. [4 points] Assume that we want to extend the basic vector addition code so that **each thread** computes **eight consecutive input elements**. (For example, thread 0 computes index 0-7, thread 1 computes index 8-15, etc.) Assume that variable `i` should be initialized with the index for the first element to be processed by a thread. What should `i` be initialized to?

```
__global__ void vecAddKernel(float* A, float* B, float* C, int n) {
      int i = ????
      for(int j = 0; j < 8; j++)
            if( (i+j) < n ) C[i+j] = A[i+j] + B[i+j];
      ...
      }
```

   int i = (blockIdx.x * blockDim.x + threadIdx.x) * 8

3. [4 points] Can the above vector add program be made faster using shared memory? Explain why or why not.

   Answer: Yes, because when using memory coalesing, we bring over groups of data from global memory (which add to latency) to shared memory, which is accessible by all threads & blocks, having memory coalesing in shared memory will significantly reduce latency during running the program.

4. [13 points] Reduction
For a Reduction kernel that operates on a total of 10000 elements using a block size of 256 threads, answer the following: (Assume each block calculates a partialSum[] as in Assignment 2)

   a. [3 point] How many steps will the reduction kernel take?

   Answer: 8

   b. [3 points] For a naïve reduction implementation, how many steps are divergent?

   Answer: 8

   c. [3 points] For an optimized reduction implementation, how many steps are divergent?

   Answer: 5

   d. [4 points] For the kernel mentioned above, the warp distribution statistics look as follows -
   W0: 0, W1: 20, W2: 20, W3: 0, W4: 20, W5: 0, W6: 0, W7: 0, W8: 20, W9: 0, W10: 0, W11: 0, W12: 0, W13: 0, W14: 0, W15: 0, W16: 20, W17: 0, W18: 0, W19: 0, W20: 0, W21: 0, W22: 0, W23: 0, W24: 0, W25: 0, W26: 0, W27: 0, W28: 0, W29: 0, W30: 0, W31: 0, W32: 300
   Why are W1, W2, W4, W8, and W16 all equal to 20?

   Answer: In Standard execution, each thread typically processes 2 elements, so a block of 256 threads to processes 10000 would require 20 blocks. Looking at per block execution, especially in part c, the active threads count drops 16, 8, 4, 2, & 1 and in each block exactly one warp executes.

Name _____ SID _____

5. [12 points] CUDA Streams.
Consider the following code snippet of a CUDA Streams implementation of Vector Add.

```
for (int i=0; i<n; i+=SegSize*2) {
(A)  cudaMemcpyAsync(d_A0, h_A+i, SegSize*sizeof(float),…, stream0);
(B)  cudaMemcpyAsync(d_B0, h_B+i, SegSize*sizeof(float),…, stream0);
(C)  vecAdd<<<SegSize/256, 256, 0, stream0>>>(d_A0, d_B0,…);
(D)  cudaMemcpyAsync(h_C+i, d_C0, SegSize*sizeof(float),…, stream0);

(E)  cudaMemcpyAsync(d_A1, h_A+i+SegSize, SegSize*sizeof(float),…, stream1);
(F)  cudaMemcpyAsync(d_B1, h_B+i+SegSize, SegSize*sizeof(float),…, stream1);
(G)  vecAdd<<<SegSize/256, 256, 0, stream1>>>(d_A1, d_B1, …);
(H)  cudaMemcpyAsync(d_C1, h_C+i+SegSize, SegSize*sizeof(float),…, stream1);
}
```

a. [4 points] Which two instructions would experience overlapped execution? Fill in the boxes below **with the letter** associated with the two instructions that overlap.

Answer 1: C Answer 2: E

b. [5 points] If you are allowed to **move only one instruction** up or down in this order to maximize pipelining in this case, which instruction would be moved below which other instruction?

Instr : D Below: F

c. [3 points] If **SegSize = 256** and **n=8192**, how many vecAdd kernels will be launched during this program?

Answer: 32

Name _____ SID _____

Multiple Choice. **Please completely fill in the circle ○ for your answer.** [12 points total]

6. [3 points] If a CUDA device's SM (streaming multiprocessor) can take up to 2048 threads and up to 32 thread blocks. Which of the following 2D block configurations would result in the greatest number of threads in each SM with the minimum number of blocks?

   ○ 64x64
   ● 32x32
   ○ 16x16
   ○ 8x8

7. [3 points] How many streams do you need to fully utilize all of the kernel and copy engines on the GPU?

   ○ 1
   ○ 2
   ● 3
   ○ 4

8. [3 points] As data is transferred from host to device using cudaMemCopy it touches various types of hardware components (memories, buses, on-chip components, etc.). Order the following hardware components in the order of being used during a cudaMemCopy H2D operation.

   ○ Pinned Memory -> Host Memory -> DMA Engine -> Global Memory
   ○ Host Memory -> DMA Engine -> Pinned Memory -> Global Memory
   ● Host Memory -> Pinned Memory -> DMA Engine -> Global Memory
   ○ Pinned Memory -> DMA Engine -> Host Memory -> Global Memory
   ○ None of the above

9. [3 points] Which of the following memory types has the fastest access speed?

   ○ L2 Cache
   ○ L1 Cache
   ● Register
   ○ Global Memory
   ○ Shared Memory

Name _____ SID _____

True or False. **For the following questions, fill the answer box with (T) True or (F) False.** [16 points total]

10. __device__ functions are only callable from the host — F

11. cudamemcpy is a synchronous process. — T

12. All blocks in a grid can be executed independent to each other in any order — T

13. A SIMT core has four independent schedulers — T

14. All cudamemcpy (host to device) copies from pinned memory — F

15. The maximum number of threads and blocks that can reside in an SM is fixed for each GPU vendor. — F

16. [6 points] You need to write a 2D kernel that operates on an image of size 500x800 (height (y-axis) x width(x-axis)). Assume your thread block size is 16x16.

    a. [2 points] What is the x-dimension of your grid size?

    Answer: 50

    800 / 16 = 50

    b. [2 points] What is the y-dimension of your grid size?

    Answer: 32

    500 / 16 = 31.25

    c. [4 point] How many warps will be divergent in this scenario?

    Answer: 0

Name _____ SID _____

17. [12 points] Active mask. Assume a warp size of 8, and the following pseudo-code will be executed. Fill in the active mask at each of the following basic block locations. Hint: the active mask for a warp size of 8 when all threads are active will be 11111111.

```
int N = 6;
__global__ function()
{
 int index = threadIdx.x; //PC = A

 if(index < N)
 {
   //PC = B

   if(index != 0)
   { //PC = C
     . . .
   }
   else
   {  //PC = D
     . . .
   }
   //PC = E

 }
 else
 { //PC = F
   . . .
 }
 __syncthreads(); //PC = G
}
```

Answer: 00111111

Answer: 00111110

Answer: 00000001

Answer: 11000000

18. [4 points] Assume that a kernel is launched with 100 thread blocks, each with 512 threads. If a variable, s_var, is declared as a shared memory variable, how many versions of s_var will be created throughout the lifetime of the execution of the kernel?

Answer: 100

Shared memory variable is allocated throughout all blocks